

against **AttributeTypeAndValue** sequences where the **type** field is **commonName** (**{joint-iso-itu-t(2) ds(5) attributeType(4) commonName(3)}**) in the **issuer** and **subject** fields, and any mention thereafter of using or matching a "Vendor ID" or "Product ID" with regards to a **Device Attestation Procedure** step SHALL rely on values obtained with that method.

For example, if a given **Product Attestation Intermediate certificate** has a **subject** field employing a particular method of encoding the VendorID and ProductID, either using only Matter-specific OIDs or only the fallback method, then it follows that a **Device Attestation Certificate** issued by the certificate authority of that Product Attestation Intermediate SHALL have the same Distinguished-Name content in its **issuer** field, so that the basic path validation algorithm works. That Device Attestation Certificate MAY however have the "fallback method" used within its **subject** field, if the Product Attestation Intermediate certificate authority is unable to encode/reflect the Matter-specific OIDs in RelativeDistinguishedName attributes within the **subject** field. The rules for whether to consider the canonical or "fallback method" for VendorID and ProductID encoding applies field by field independently for each instance of **subject** or **issuer** field found in certificates within the DAC chain.

6.2.2.3. Device Attestation Certificate (DAC)

The attributes in a DAC include:

```
Certificate ::= SEQUENCE {
    tbsCertificate      DACTBSCertificate,
    signatureAlgorithm  AlgorithmIdentifier,
    signatureValue      BIT STRING }
```

```
DACTBSCertificate ::= SEQUENCE {
    version             INTEGER ( v3(2) ),
    serialNumber        INTEGER,
    signature           MatterSignatureIdentifier,
    issuer              MatterPAName,
    validity            Validity,
    subject             MatterDACName,
    subjectPublicKeyInfo SEQUENCE {
        algorithm        OBJECT IDENTIFIER(id-x962-prime256v1),
        subjectPublicKey  BIT STRING },
    extensions          DACExtensions }
```

```
DACExtensions ::= SEQUENCE {
    basicConstraint Extension({extnID id-ce-basicConstraints, critical TRUE, extnValue
BasicConstraints {cA FALSE} })),
    keyUsage Extension({extnID id-ce-keyUsage, critical TRUE, extnValue
KeyUsage({digitalSignature}})),
    authorityKeyIdentifier Extension({extnID id-ce-authorityKeyIdentifier}),
    subjectKeyIdentifier Extension({extnID id-ce-subjectKeyIdentifier}),
```